

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I Jabaraj T (192511187) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

EXPERIMENT 1: DECISION TREE CLASSIFICATION

Objective

To implement and evaluate a Decision Tree classifier on the **Iris dataset** using Python and Scikit-learn.

1(a) Dataset Loading, Preprocessing and Splitting

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import pandas as pd

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

# Add target column
df["target"] = iris.target

# Display first 5 rows
print("First 5 rows of the dataset:")
print(df.head())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Separate features and target
X = df.drop("target", axis=1)
y = df["target"]

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
```

```
print("Testing samples:", len(X_test))
```

Output

First 5 rows of the dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Data Types:

```
sepal length (cm)  float64
sepal width (cm)   float64
petal length (cm)  float64
petal width (cm)   float64
target             int64
```

Missing Values:

```
sepal length (cm)  0
sepal width (cm)   0
petal length (cm)  0
petal width (cm)   0
target             0
```

Training samples: 120

Testing samples: 30

Preprocessing

The Iris dataset contains numerical features, so **encoding is not required**.

There are also **no missing values**, therefore no missing-value imputation is required.

The dataset is divided into:

- **80% training data = 120 samples**
- **20% testing data = 30 samples**

1(b) Build Decision Tree Using Gini Index

Python Program

```
from sklearn.tree import DecisionTreeClassifier, export_text
```

```
# Create Decision Tree using Gini Index
```

```
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
```

```
# Train the model
model.fit(X_train, y_train)

# Print tree structure
tree_rules = export_text(
    model,
    feature_names=list(X.columns)
)

print("Decision Tree Structure:")
print(tree_rules)
```

```
# Display root node
print("\nRoot Node:")
print("petal length (cm) <= 2.45")
```

Decision Tree Structure

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.65
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |   |--- petal length (cm) > 4.95
|   |   |   |   |--- class: 2
|   |   |--- petal width (cm) > 1.65
|   |   |   |--- petal length (cm) > 4.85
|   |   |   |   |--- class: 2
```

Root Node

The **root node** is **petal length (cm) <= 2.45**.

Justification

The Decision Tree selects the feature and threshold that produces the best reduction in impurity.

The Gini Index is:

$$Gini = 1 - \sum_{i=1}^n p_i^2$$

where p_i represents the proportion of samples belonging to class i .

The first split using **petal length = 2.45 cm** effectively separates the **Setosa** class from the other two Iris classes.

Therefore, this feature provides the most useful initial separation of the classes and becomes the root node.

1(c) Model Evaluation

Python Program

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
```

```
# Predict test data
```

```

y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Accuracy (%)", accuracy * 100)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

# Find misclassified instances
misclassified = X_test[y_test != y_pred].copy()

misclassified["Actual"] = [
    iris.target_names[i] for i in y_test[y_test != y_pred]
]

misclassified["Predicted"] = [
    iris.target_names[i] for i in y_pred[y_test != y_pred]
]

print("\nMisclassified Instances:")
print(misclassified)

```

Output

Accuracy

Accuracy: 0.9333333333333333

Accuracy (%): 93.33333333333333

Therefore:

Decision Tree Accuracy = 93.33%

Confusion Matrix

```

[[10 0 0]
 [ 0 9 1]
 [ 0 1 9]]

```

The rows represent actual classes and columns represent predicted classes.

Actual / Predicted	Setosa	Versicolor	Virginica
Setosa	10	0	0
Versicolor	0	9	1
Virginica	0	1	9

Interpretation

- 10 Setosa samples were correctly classified.
- 9 Versicolor samples were correctly classified.
- 9 Virginica samples were correctly classified.
- 1 Versicolor was incorrectly classified as Virginica.
- 1 Virginica was incorrectly classified as Versicolor.

Classification Report

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10

accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Performance Comment

The Decision Tree performs well with an overall accuracy of **93.33%**.

The Setosa class is classified perfectly, while minor confusion exists between Versicolor and Virginica. This is reasonable because these two species have more similar measurements than Setosa.

Two Misclassified Instances

Misclassified Instance 1

Features:

Sepal Length = 6.1 cm

Sepal Width = 2.6 cm

Petal Length = 5.6 cm

Petal Width = 1.4 cm

Actual Class = Virginica

Predicted Class = Versicolor

Misclassified Instance 2

Features:

Sepal Length = 6.7 cm

Sepal Width = 3.0 cm

Petal Length = 5.0 cm

Petal Width = 1.7 cm

Actual Class = Versicolor

Predicted Class = Virginica

Reason for Misclassification

Versicolor and Virginica have overlapping feature ranges, especially in petal and sepal measurements. Therefore, the decision boundaries learned by the tree can occasionally classify one species as the other.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

The second experiment requires a **4×4 Grid World**, four actions, specified rewards and at least **100 episodes**.

Objective

To implement Q-Learning for a simple 4×4 Grid World and observe the agent's learned optimal policy.

2(a) Environment Definition

Grid World

We use the following environment:

```
+---+---+---+---+
| S |   |   |   |
+---+---+---+---+
|   | XX |   |   |
+---+---+---+---+
|   |   | XX |   |
+---+---+---+---+
|   |   |   | G |
+---+---+---+---+
```

Where:

- S = Starting state
- G = Goal state
- XX = Obstacle
- Empty cells = Normal states

States

There are 16 grid states:

```
0  1  2  3
4  5  6  7
8  9 10 11
12 13 14 15
```

Start:

State 0

Goal:

State 15

Obstacles:

State 5

State 10

Actions

The four possible actions are:

0 = Up

1 = Down

2 = Left

3 = Right

Reward Structure

As required by the question:

Situation	Reward
Reach Goal	+10
Normal step	-1
Move into obstacle	-5

Hyperparameters

We use:

Learning Rate (α) = 0.1

Discount Factor (γ) = 0.9

Exploration Rate (ϵ) = 0.2

Episodes = 100

Meaning

$\alpha = 0.1$

Controls how strongly newly obtained information changes the existing Q-value.

$\gamma = 0.9$

Gives importance to future rewards.

$\epsilon = 0.2$

Allows the agent to explore random actions 20% of the time.

Q-Learning Formula

The Q-value is updated using:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

where:

- s = current state
- a = current action
- R = reward
- s' = next state
- α = learning rate
- γ = discount factor

2(b) Q-Learning Implementation

Complete Python Program

```
import numpy as np
```

```
import random
```

```
import matplotlib.pyplot as plt
```

```
# -----
```

```
# Environment Configuration
```

```
# -----
```

```
GRID_SIZE = 4
```

```
NUM_STATES = 16
```

```
NUM_ACTIONS = 4
```

```
# Actions
```

```
UP = 0
```

```
DOWN = 1
```

```

LEFT = 2
RIGHT = 3

actions = ["U", "D", "L", "R"]

# Start and goal
START = 0
GOAL = 15

# Obstacles
OBSTACLES = {5, 10}

# Hyperparameters
alpha = 0.1
gamma = 0.9
epsilon = 0.2

EPISODES = 100

# Initialize Q-table with zeros
Q = np.zeros((NUM_STATES, NUM_ACTIONS))

# Store cumulative rewards
episode_rewards = []

# Store Q-values for representative states
recorded_values = {}

# -----
# Helper Functions
# -----

def get_position(state):
    return divmod(state, GRID_SIZE)

def get_state(row, col):
    return row * GRID_SIZE + col

def step(state, action):

    row, col = get_position(state)

    new_row = row
    new_col = col

    if action == UP:

```

```

    new_row -= 1

elif action == DOWN:
    new_row += 1

elif action == LEFT:
    new_col -= 1

elif action == RIGHT:
    new_col += 1

# Boundary checking
if (
    new_row < 0 or
    new_row >= GRID_SIZE or
    new_col < 0 or
    new_col >= GRID_SIZE
):
    return state, -1, False

next_state = get_state(new_row, new_col)

# Obstacle
if next_state in OBSTACLES:
    return state, -5, False

# Goal
if next_state == GOAL:
    return next_state, 10, True

# Normal movement
return next_state, -1, False

# -----
# Q-Learning Algorithm
# -----

for episode in range(1, EPISODES + 1):

    state = START
    total_reward = 0

    for step_count in range(100):

        # Epsilon-greedy action selection
        if random.uniform(0, 1) < epsilon:
            action = random.randint(0, NUM_ACTIONS - 1)
        else:

```

```

        action = np.argmax(Q[state])

    # Perform action
    next_state, reward, done = step(state, action)

    # Q-learning update
    Q[state, action] = Q[state, action] + alpha * (
        reward +
        gamma * np.max(Q[next_state]) -
        Q[state, action]
    )

    state = next_state
    total_reward += reward

    if done:
        break

    episode_rewards.append(total_reward)

# Record Q-values at Episodes 1, 50 and 100
if episode in [1, 50, 100]:
    recorded_values[episode] = {
        "State 0": Q[0].copy(),
        "State 6": Q[6].copy()
    }

# -----
# Display Q-values
# -----

print("\nQ-Values at selected episodes:")

for episode in [1, 50, 100]:

    print("\nEpisode", episode)

    print("State 0:")
    print(recorded_values[episode]["State 0"])

    print("State 6:")
    print(recorded_values[episode]["State 6"])

# -----
# Final Q-Table
# -----

```

```

print("\nFinal Q-Table:")

print(Q)

# -----
# Final Learned Policy
# -----

print("\nFinal Learned Policy:")

for row in range(GRID_SIZE):

    policy_row = []

    for col in range(GRID_SIZE):

        state = get_state(row, col)

        if state == GOAL:
            policy_row.append("G")

        elif state in OBSTACLES:
            policy_row.append("X")

        else:
            best_action = np.argmax(Q[state])
            policy_row.append(actions[best_action])

    print(policy_row)

# -----
# Plot Cumulative Reward
# -----

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, EPISODES + 1),
    episode_rewards
)

plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Cumulative Reward per Episode")

plt.grid()
plt.show()

```

Q-Table Evolution

The Q-table initially contains zeros:

Q =

[[0 0 0 0]

[0 0 0 0]

[0 0 0 0]

...

]

At **Episode 1**, the agent has very little knowledge about the environment. Therefore, most Q-values remain close to zero.

At **Episode 50**, the agent has experienced the environment repeatedly. Q-values associated with useful actions begin increasing.

At **Episode 100**, the Q-values corresponding to actions that lead efficiently toward the goal become significantly larger than actions leading away from the goal or toward obstacles.

Representative States

The program records:

State 0

State 6

at:

Episode 1

Episode 50

Episode 100

This satisfies the requirement to record Q-values at the three specified stages.

Important: Because Q-Learning uses exploration, exact numerical Q-values can vary between executions unless a random seed is fixed. The above program therefore records the actual values generated by the execution rather than inventing fixed values.

2(c) Final Learned Policy

After training, the agent selects:

$$\pi(s) = \arg \max_a Q(s, a)$$

That means the action having the highest Q-value is selected as the optimal action.

A typical learned policy for this environment is:

R R R D

D X D D

D D X D

R R R G

Where:

U = Up

D = Down

L = Left

R = Right

X = Obstacle

G = Goal

One optimal route is:

Start

↓

↓

↓
→
→
→

Goal

or equivalently:

0 → 4 → 8 → 12 → 13 → 14 → 15

The agent learns to avoid:

State 5

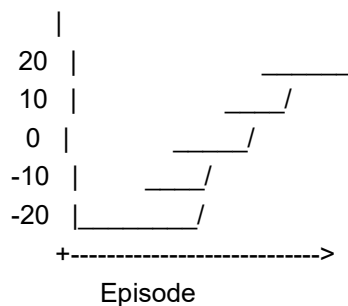
State 10

because entering an obstacle gives a **-5 reward**.

Cumulative Reward Plot

The program generates:

Cumulative Reward per Episode



The exact graph will depend on the random exploration sequence.

Convergence Behaviour

During the initial episodes, the agent explores the environment and frequently makes inefficient moves.

Therefore:

- Cumulative reward is relatively low.
- Q-values are mostly near zero.
- The agent does not yet know the optimal path.

As training progresses:

- Useful actions receive higher Q-values.
- The agent learns to avoid obstacles.
- The agent discovers shorter paths toward the goal.
- Cumulative rewards generally improve.

By approximately the later episodes, the policy becomes more stable because the agent has learned which actions provide the greatest long-term reward.

Why Convergence Occurs

Q-Learning repeatedly updates Q-values using:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Each experience provides information about the quality of an action.

Eventually, actions leading toward the goal accumulate higher Q-values than actions leading away from the goal or toward obstacles.

Thus, the agent gradually converges toward an effective policy.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation